

8. Semana 8. Método de Elementos Finitos I

Objetivo de la cátedra. Comprender la formulación matemática y computacional del método de elementos finitos (FEM): la transición de un problema de valores en la frontera continuo a un sistema algebraico discreto, el papel de la discretización espacial y las funciones de forma, el ensamble matricial y la imposición de condiciones de frontera. El eje aplicado es el módulo `STATIC STRUCTURAL` de `ANSYS WORKBENCH`; la generación de malla se ilustra primero en Python (Google Colab) para hacer explícitas las decisiones de mallado que en `ANSYS` quedan ocultas tras la automatización.

Temario

- Introducción al FEM.
- Discretización espacial.
- Generación de mallas.
- Elementos y nodos.
- Funciones de forma.
- Ensamble matricial.
- Condiciones de frontera.

8.1. Introducción al FEM

El FEM es un método de discretización para resolver de forma aproximada ecuaciones diferenciales parciales (EDP) sobre dominios de geometría arbitraria. Su idea central es sustituir el problema continuo, con infinitos grados de libertad, por un sistema algebraico finito construido sobre una partición del dominio en subregiones simples (los *elementos*), sobre las cuales la solución se aproxima mediante polinomios definidos a trozos.

Para el análisis estructural estático lineal —el caso del módulo `STATIC STRUCTURAL`— la EDP gobernante es la ecuación de equilibrio de la elastostática. En su *forma fuerte*, para un cuerpo que ocupa el dominio $\Omega \subset \mathbb{R}^d$ ($d = 2, 3$):

$$\nabla \cdot \boldsymbol{\sigma} + \mathbf{b} = \mathbf{0} \quad \text{en } \Omega, \quad (216)$$

sujeta a la ley constitutiva y a la relación cinemática de pequeñas deformaciones:

$$\boldsymbol{\sigma} = \mathbf{D} : \boldsymbol{\varepsilon}, \quad (217)$$

$$\boldsymbol{\varepsilon} = \frac{1}{2} (\nabla \mathbf{u} + \nabla \mathbf{u}^T), \quad (218)$$

donde $\boldsymbol{\sigma}$ es el tensor de esfuerzos, $\boldsymbol{\varepsilon}$ el de deformaciones, $\mathbf{u}$ el campo de desplazamientos,

$\mathbf{b}$ las fuerzas de cuerpo y $\mathbf{D}$ el tensor constitutivo elástico (para material isótropo, función de E y ν). Las condiciones de frontera son de tipo Dirichlet (desplazamiento prescrito $\mathbf{u} = \bar{\mathbf{u}}$ en Γ_u) y Neumann (tracción prescrita $\boldsymbol{\sigma} \cdot \mathbf{n} = \bar{\mathbf{t}}$ en Γ_t).

El FEM no opera sobre la forma fuerte, sino sobre su *forma débil* o variacional. Multiplicando Ecuación (216) por una función de prueba admisible $\mathbf{w}$ (con $\mathbf{w} = \mathbf{0}$ en Γ_u) e integrando por partes se obtiene el principio de los trabajos virtuales:

$$\int_{\Omega} \boldsymbol{\varepsilon}(\mathbf{w}) : \mathbf{D} : \boldsymbol{\varepsilon}(\mathbf{u}) \, d\Omega = \int_{\Omega} \mathbf{w} \cdot \mathbf{b} \, d\Omega + \int_{\Gamma_t} \mathbf{w} \cdot \bar{\mathbf{t}} \, d\Gamma. \quad (219)$$

La forma débil (219) exige solo derivadas de primer orden de $\mathbf{u}$ (frente al segundo orden de la forma fuerte), lo que permite usar aproximaciones polinómicas continuas a trozos (C^0). El método de *Galerkin* consiste en elegir las funciones de prueba $\mathbf{w}$ del mismo espacio discreto que la solución aproximada $\mathbf{u}_h$.

Tabla 41: Comparación de métodos de discretización para EDP.

Aspecto	Diferencias finitas	Elementos finitos	Volumen finito
Formulación	Forma fuerte	Forma débil (Galerkin)	Balance integral
Geometría	Mallas regulares	Geometría arbitraria	Geometría arbitraria
Frontera curva	Difícil	Natural (iso- paramétrica)	Natural
Uso dominante	Térmico/CFD simple	Estructural/mecánico	CFD

La Tabla 41 sitúa al FEM frente a otras técnicas. Su ventaja decisiva para el diseño mecánico es el tratamiento natural de geometrías con fronteras curvas mediante la formulación isoparamétrica (section 8.4), razón por la cual es el estándar en análisis estructural.

8.2. Discretización espacial

La discretización espacial es la partición del dominio Ω en un conjunto finito de elementos $\{\Omega_e\}_{e=1}^{n_{el}}$ que no se solapan y cuya unión aproxima Ω :

$$\Omega \approx \Omega_h = \bigcup_{e=1}^{n_{el}} \Omega_e, \quad \Omega_i \cap \Omega_j = \emptyset \quad (i \neq j). \quad (220)$$

La malla debe ser *conforme*: dos elementos adyacentes comparten una arista (2D) o cara (3D) completa, sin nodos colgantes. El parámetro característico es el tamaño de elemento h (longitud representativa de arista). La teoría de aproximación establece, para elementos

de orden polinómico p y solución suficientemente regular, la cota de error *a priori*:

$$\|\mathbf{u} - \mathbf{u}_h\|_E \leq C h^p, \quad (221)$$

donde $\|\cdot\|_E$ es la norma energética y C una constante independiente de h . La [Ecuación \(221\)](#) justifica las dos vías de mejora de la precisión: refinamiento h (reducir el tamaño de elemento) y refinamiento p (elevar el orden polinómico).

En la práctica no se conoce la solución exacta, por lo que la convergencia se verifica comparando mallas sucesivamente refinadas. Se adopta el criterio de convergencia de malla empleado en el hilo de validación DIC–FEA del curso:

$$e_{\text{malla}} = \frac{|\varepsilon_{xx}^{(h)} - \varepsilon_{xx}^{(h/2)}|}{|\varepsilon_{xx}^{(h/2)}|} \leq 2\%, \quad (222)$$

es decir, la malla se considera convergida cuando la cantidad de interés (aquí la deformación ε_{xx}) no cambia más del 2 % al reducir a la mitad el tamaño de elemento. Este error de discretización e_{num} es una de las cuatro contribuciones al error total en una comparación experimento–modelo y debe mantenerse por debajo del 2 % para que no domine el presupuesto.

Enlace metrológico. En una validación experimental por correlación digital de imágenes (DIC), el tamaño de elemento FEA no debe elegirse arbitrariamente fino: conviene generar una malla *convergada* ($h \leq 1$ mm) para obtener la solución “verdadera” del modelo y una malla *compatible* ($h \approx L_{\text{VSG}}$, el tamaño del subconjunto virtual de deformación del DIC) para comparar punto a punto. Si ambas coinciden, los gradientes son suaves y la comparación es directa; si difieren, el gradiente es agudo y debe considerarse el sesgo de suavizado del DIC.

8.3. Elementos y nodos

El elemento es la unidad geométrica y de interpolación; los *nodos* son los puntos donde se definen los grados de libertad (GDL) —las incógnitas del problema. En análisis estructural 2D cada nodo porta dos GDL de desplazamiento (u_x, u_y); en 3D, tres (u_x, u_y, u_z); los elementos de cáscara y viga añaden GDL rotacionales. El número total de GDL del modelo es

$$n_{\text{GDL}} = d_{\text{nodo}} \cdot n_{\text{nodos}}, \quad (223)$$

con d_{nodo} los GDL por nodo. La [Ecuación \(223\)](#) fija el tamaño del sistema algebraico y, por tanto, el costo computacional.

La distinción fundamental es entre elementos *lineales* (solo nodos de esquina, fun-

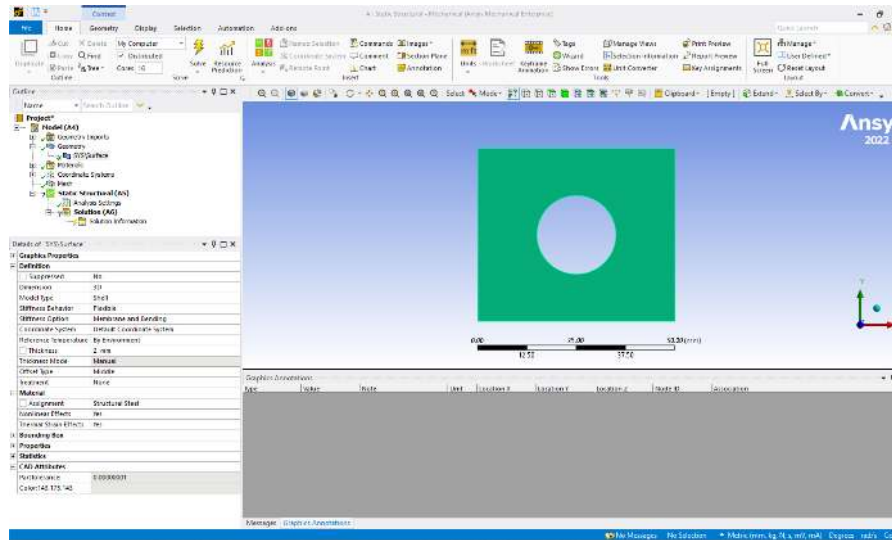


Figura 55: Definición de la geometría 2D en ANSYS. La placa con barreno central reproduce el dominio del ejemplo computacional (Ec. 220).

ciones de forma de primer orden) y *cuadráticos* (con nodos intermedios en las aristas, segundo orden). La Figura 56 muestra las cuatro configuraciones canónicas en 2D y su conteo de GDL. Los elementos cuadráticos representan con exactitud fronteras curvas (la arista es una parábola) y capturan mejor los gradientes de esfuerzo, a costa de más GDL por elemento.

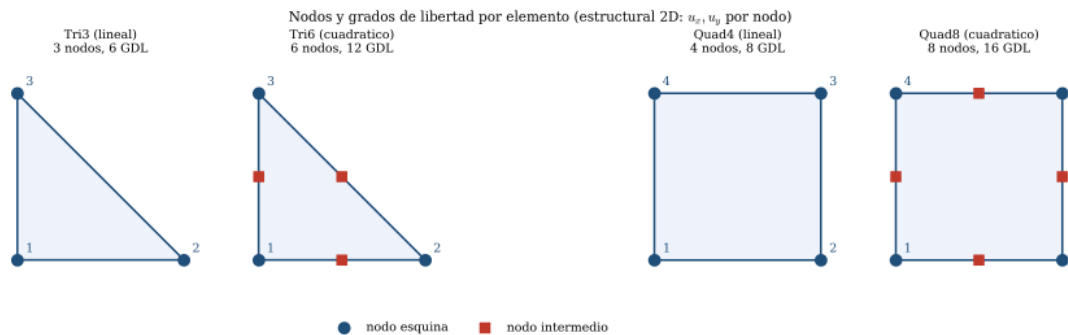


Figura 56: Nodos y grados de libertad de los elementos 2D canónicos. Los nodos de esquina (azul) definen los elementos lineales Tri3 y Quad4; los nodos intermedios (rojo) los promueven a cuadráticos Tri6 y Quad8. Cada nodo aporta dos GDL (u_x, u_y) según Ec. 223.

La Tabla 42 lista elementos de uso frecuente. La nomenclatura CalculiX/Abaqus equivalente para el tetraedro cuadrático es C3D10, el elemento por defecto en el modelo de validación de flexión a tres puntos del curso. Una regla operativa: los elementos cuadráticos de tetraedro (SOLID187, C3D10) son la opción robusta para geometrías complejas mallables solo con tetraedros, mientras que los hexaedros lineales o cuadráticos se prefieren donde la geometría admite malla mapeada o barrida.

Tabla 42: Elementos estructurales representativos de la biblioteca de ANSYS.

Elemento	Dim.	Nodos	Orden	Uso típico
PLANE182	2D	4	Lineal	Tensión/deformación plana, axisimétrico
PLANE183	2D	6/8	Cuadr.	Ídem, fronteras curvas y gradientes altos
SOLID185	3D	8	Lineal	Sólido hexaédrico general
SOLID186/187	3D	20/10	Cuadr.	Sólido hexa/tetra de segundo orden
SHELL181	3D	4	Lineal	Cáscaras delgadas (6 GDL/nodo)

Consistencia de GDL en interfaces. Al unir elementos de distinto tipo se debe cuidar la compatibilidad de GDL. Un SOLID185 (3 GDL de traslación) unido a un SHELL181 (3 de traslación + 3 rotacionales) no transmite momentos a través de la interfaz, pues el sólido carece de GDL rotacionales. La continuidad exige que ambos elementos compartan el mismo número y tipo de GDL en los nodos comunes.

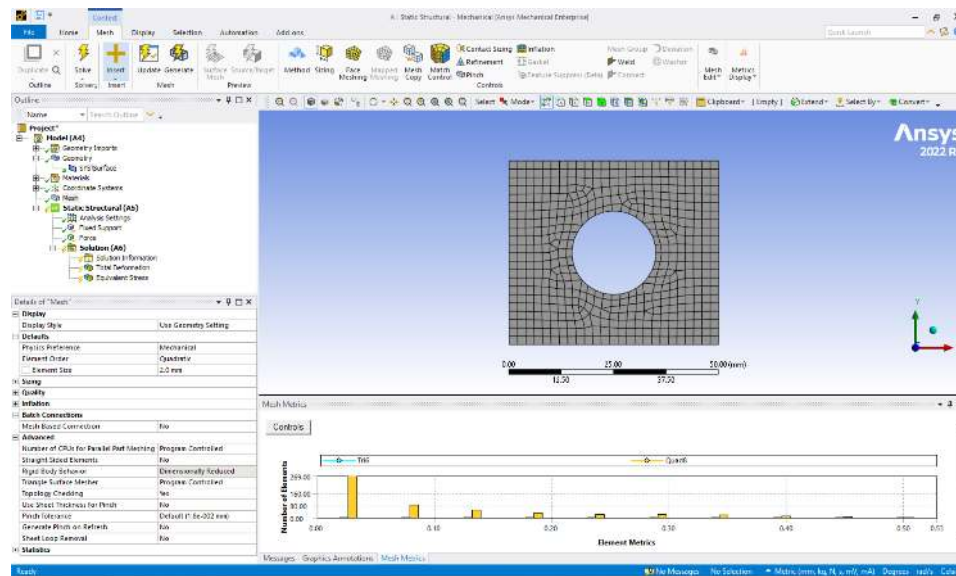


Figura 57: Control del orden del elemento en ANSYS: *Dropped* produce elementos lineales; *Kept*, cuadráticos con nodos intermedios, elevando los GDL según Ec. 223.

8.4. Funciones de forma

Dentro de cada elemento, el campo de desplazamientos se interpola a partir de los valores nodales $\mathbf{u}_i$ mediante las *funciones de forma* $N_i(\boldsymbol{\xi})$:

$$\mathbf{u}_h(\boldsymbol{\xi}) = \sum_{i=1}^{n_{en}} N_i(\boldsymbol{\xi}) \mathbf{u}_i, \quad (224)$$

donde n_{en} es el número de nodos del elemento y $\boldsymbol{\xi}$ las coordenadas naturales (paramétricas). Las funciones de forma deben satisfacer:

1. **Delta de Kronecker:** $N_i(\boldsymbol{\xi}_j) = \delta_{ij}$ (vale 1 en su nodo y 0 en los demás).
2. **Partición de la unidad:** $\sum_{i=1}^{n_{en}} N_i(\boldsymbol{\xi}) = 1$, condición necesaria para representar exactamente un campo constante (movimiento de cuerpo rígido).
3. **Compleitud:** capacidad de reproducir polinomios hasta el orden requerido para la convergencia.

Para el elemento cuadrilátero bilineal (Q4), en coordenadas naturales $(\xi, \eta) \in [-1, 1]^2$:

$$N_i(\xi, \eta) = \frac{1}{4} (1 + \xi_i \xi)(1 + \eta_i \eta), \quad i = 1, \dots, 4, \quad (225)$$

con (ξ_i, η_i) las coordenadas nodales. La [Figura 58](#) muestra las funciones lineales y cuadráticas en 1D y la superficie bilineal $N_1(\xi, \eta)$ del Q4. La verificación numérica del script confirma la partición de la unidad para la base cuadrática 1D y la consistencia de la base Q4.

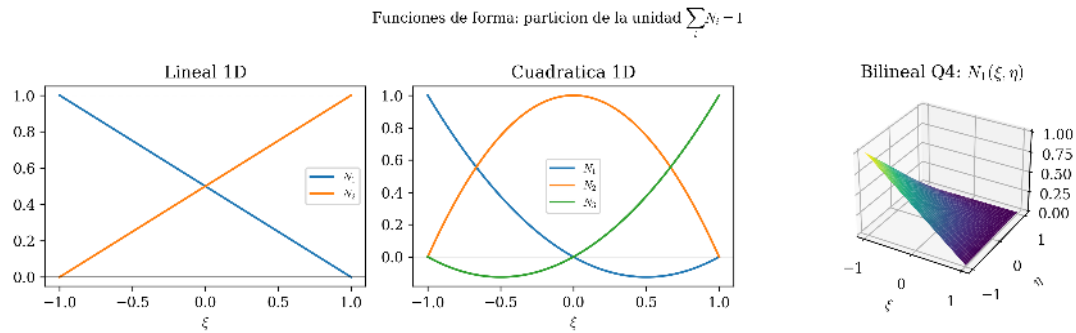


Figura 58: Funciones de forma. Izquierda: base lineal 1D; centro: base cuadrática 1D de Lagrange; derecha: función bilineal $N_1(\xi, \eta)$ del elemento Q4. Todas satisfacen la partición de la unidad $\sum_i N_i = 1$.

Formulación isoparamétrica. La misma base N_i interpola tanto el campo incógnita (224) como la geometría del elemento:

$$\mathbf{x}(\boldsymbol{\xi}) = \sum_{i=1}^{n_{en}} N_i(\boldsymbol{\xi}) \mathbf{x}_i. \quad (226)$$

Esta identidad —el concepto isoparamétrico— es la que permite mapear un elemento de referencia recto a un elemento distorsionado con aristas curvas, y es la base del mallado de fronteras curvas. La transformación entre el espacio natural y el físico se cuantifica mediante el jacobiano $\mathbf{J} = \partial \mathbf{x} / \partial \boldsymbol{\xi}$; su determinante debe ser positivo en todo el elemento (un jacobiano negativo indica un elemento invertido y provoca fallo de mallado). La razón jacobiana es, de hecho, una de las métricas de calidad de malla (section 8.6).

8.5. Ensamble matricial

Sustituyendo la interpolación (224) en la forma débil (219) y aplicando Galerkin, cada elemento contribuye con una *matriz de rigidez elemental*:

$$\mathbf{K}_e = \int_{\Omega_e} \mathbf{B}^T \mathbf{D} \mathbf{B} d\Omega, \quad (227)$$

donde $\mathbf{B} = \partial \mathbf{N}$ es la matriz deformación–desplazamiento (derivadas de las funciones de forma) y $\mathbf{D}$ el tensor constitutivo. La integral (227) se evalúa numéricamente por cuadratura de Gauss sobre el elemento de referencia, empleando el jacobiano de (226) para el cambio de variable.

Las contribuciones elementales se *ensamblan* en la matriz de rigidez global $\mathbf{K}$ y el vector de fuerzas $\mathbf{F}$ mediante el operador de ensamble $\bigwedge$ (mapeo de GDL locales a globales):

$$\mathbf{K} = \bigwedge_{e=1}^{n_{el}} \mathbf{K}_e, \quad \mathbf{F} = \bigwedge_{e=1}^{n_{el}} \mathbf{F}_e, \quad (228)$$

donde $\bigwedge$ denota el operador de ensamble (suma con mapeo de GDL locales a globales). El sistema algebraico resultante es lineal:

$$\mathbf{K} \mathbf{U} = \mathbf{F}, \quad (229)$$

con $\mathbf{U}$ el vector global de desplazamientos nodales ($\dim \mathbf{U} = n_{\text{GDL}}$). La matriz $\mathbf{K}$ es simétrica, definida positiva (tras imponer las condiciones de frontera que eliminan los modos de cuerpo rígido) y *dispersa*: cada nodo solo interactúa con sus vecinos inmediatos, por lo que $\mathbf{K}$ tiene estructura de banda. Esta dispersión es la que hace tratables modelos con millones de GDL y motiva el uso de solvers dispersos (directos tipo Cholesky o iterativos tipo PCG).

Tabla 43: Secuencia de ensamble y solución en STATIC STRUCTURAL.

Paso	Operación	Resultado
1	Evaluar $\mathbf{K}_e$ por cuadratura (Ec. 227)	Matrices elementales
2	Mapear GDL locales $\rightarrow$ globales	Tabla de conectividad
3	Ensamblar $\mathbf{K}$, $\mathbf{F}$ (Ec. 228)	Sistema global disperso
4	Imponer condiciones de frontera	Sistema reducido no singular
5	Resolver $\mathbf{KU} = \mathbf{F}$ (Ec. 229)	Desplazamientos nodales
6	Recuperar ϵ , σ (post)	Campos de deformación y esfuerzo

8.6. Generación de mallas

La generación de malla es la etapa donde se materializa la discretización (220). La calidad de la malla condiciona directamente la precisión (a través de la razón jacobiana en (227)) y la convergencia del solver. Un mallado inadecuado —elementos distorsionados, transiciones bruscas, resolución insuficiente en zonas de gradiente alto— degrada la solución aun con una formulación correcta. Para hacer explícitas estas decisiones, el ejemplo computacional (Anexo A) genera y compara, sobre una misma geometría con frontera curva, las estrategias de mallado disponibles en ANSYS Workbench.

Geometría de referencia. Placa cuadrada de 50×50 mm con barreno circular central de radio 10 mm (Figura 59). Es el *benchmark* clásico de concentración de esfuerzos (problema de Kirsch), y su frontera curva obliga a decidir cómo mallar la vecindad del barreno. El material de referencia es aluminio 6061-T6 ($E = 68.9$ GPa, $\nu = 0.33$).

Estrategias de mallado. Se ilustran cuatro:

1. **Malla libre triangular** (Figura 60a). Algoritmo de avance frontal/Delaunay; llena dominios arbitrarios sin restricciones topológicas. Es la opción por defecto más robusta (*Patch Conforming Tetrahedron* en 3D), pero genera más elementos y mayor dispersión de calidad.
2. **Malla de cuadriláteros** (Figura 60b). Recombinación de triángulos en cuadriláteros (algoritmo Blossom). Menos elementos y, en general, mejor precisión por GDL que los triángulos lineales, que son excesivamente rígidos.
3. **Malla mapeada estructurada** (Figura 61). Malla transfinita sobre un sector anular en torno al barreno: topología regular tipo rejilla, con conectividad predecible y calidad óptima. Es el análogo del *Mapped Face Meshing / Sweep* de ANSYS.
4. **Malla con sesgo** (Figura 62). La misma malla mapeada, con *sesgo* (bias) radial

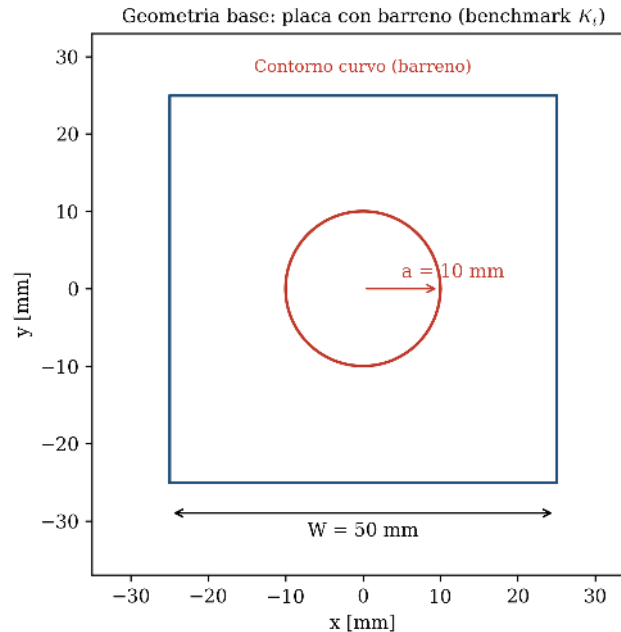


Figura 59: Geometría base del ejemplo: placa con barreno central. La frontera curva (rojo) es la región crítica donde se concentran los esfuerzos y donde el mallado debe refinarse.

que concentra nodos hacia el barreno, donde el gradiente de esfuerzo es máximo. Es el *Bias Factor* de ANSYS.

Métricas de calidad. La calidad de cada elemento se cuantifica con métricas geométricas. Las principales en ANSYS son:

- **Skewness (asimetría):** desviación respecto al elemento equiangular ideal. Rango $[0, 1]$; 0 es perfecto. Para un triángulo el ideal tiene ángulos de 60° ; para un cuadrilátero, de 90° .
- **Orthogonal Quality:** ortogonalidad de las caras. Rango $[0, 1]$; 1 es óptimo, 0 pésimo.
- **Aspect Ratio:** razón entre la arista mayor y la menor; 1 es ideal, valores altos indican elementos estirados.
- **Jacobian Ratio:** uniformidad del mapeo isoparamétrico; detecta elementos invertidos o excesivamente distorsionados.

Los resultados numéricos del ejemplo (Tabla 45) cuantifican los compromisos. La malla mapeada logra la mejor calidad (skewness máximo 0.033) con el menor número de GDL (386), frente a la malla triangular libre (996 GDL, skewness máximo 0.385) y la de cuadriláteros (842 GDL, skewness máximo 0.526). El sesgo, aplicado sobre la malla mapeada, mantiene el skewness (0.033) pero eleva la razón de aspecto de 1.92 a 2.30: el compromiso esperado, pues concentrar nodos radialmente estira los elementos. Este

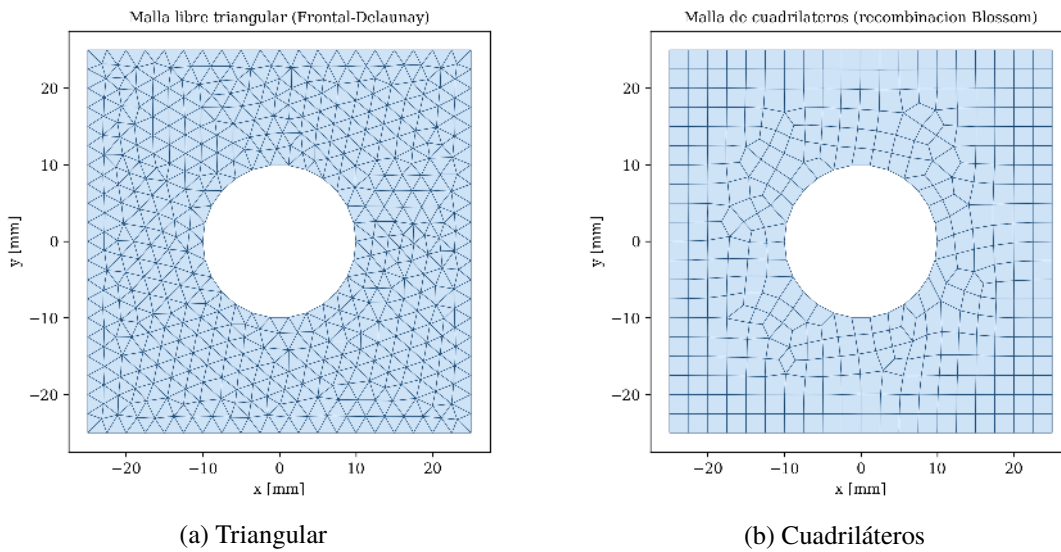


Figura 60: Mallas libres sobre la placa con barreno. (a) Triangular (Frontal–Delaunay). (b) Cuadriláteros por recombinación. Ambas resuelven la frontera curva sin intervención del usuario, a costa de mayor número de GDL.

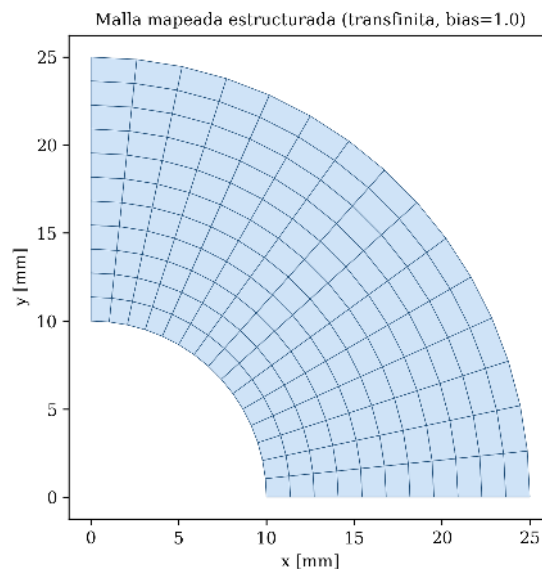


Figura 61: Malla mapeada estructurada (transfinita) sobre el sector anular en torno al barreno. La topología regular garantiza calidad uniforme; equivale al *Mapped Face Meshing* de ANSYS.

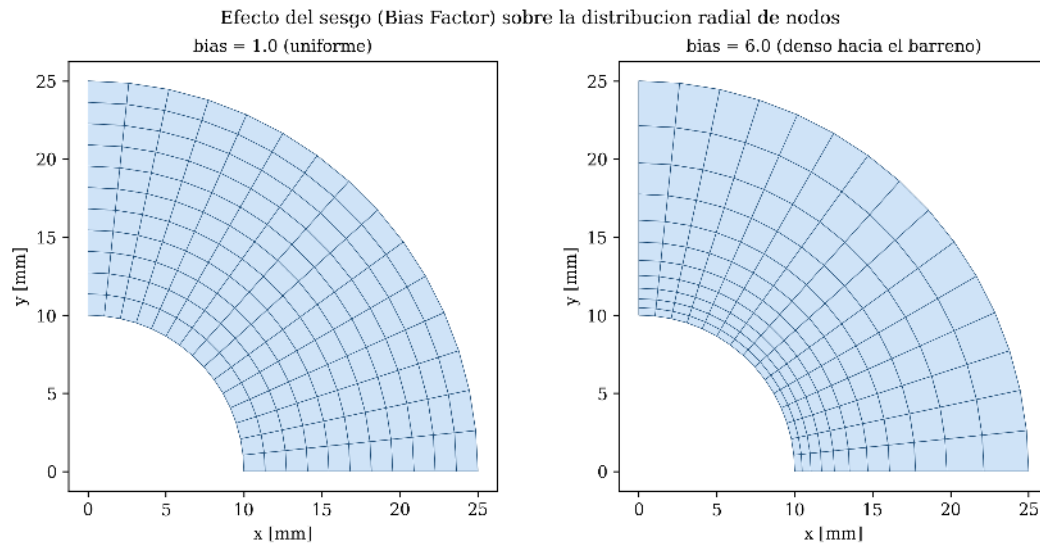


Figura 62: Efecto del sesgo (Bias Factor). Izquierda: distribución radial uniforme. Derecha: sesgo que concentra nodos hacia el barreno. El sesgo mejora la resolución del gradiente sin aumentar el número total de nodos.

Tabla 44: Límites de forma de ANSYS (opción Standard Mechanical): un valor fuera de rango produce fallo de mallado.

Métrica	Límite	Interpretación
Aspect Ratio	— (métrica informativa)	Elementos estirados
Jacobian Ratio (2D)	> 10	Mapeo degenerado
Jacobian Ratio (3D)	> 40	Mapeo degenerado
Orthogonal Quality	≤ 0.01–0.05	Cara casi degenerada
Skewness	(recomendado < 0.9)	Elemento asimétrico

es el punto pedagógico central —*el sesgo afecta la relación de aspecto, no la asimetría angular*— y la razón por la cual el Bias Factor debe usarse con mesura.

Tabla 45: Comparación cuantitativa de estrategias de mallado sobre la placa con barreno (salida real del script del Anexo A).

Estrategia	Nodos	Elem.	GDL	Skew. máx.	AR máx.
Triangular libre	498	890	996	0.385	1.66
Cuadriláteros	421	368	842	0.526	2.15
Mapeada uniforme	193	165	386	0.033	1.92
Mapeada + sesgo (BF=6)	193	165	386	0.033	2.30

El estudio de refinamiento (Figura 63) ilustra el crecimiento del costo: al reducir h de 5 a 1.2 mm, los GDL crecen de 276 a 3884 (factor ~ 14), mientras el skewness medio mejora de 0.136 a 0.049. La convergencia (Ec. 222) exige refinar hasta que la cantidad de interés se estabilice, no indefinidamente: refinar más allá de la convergencia solo incrementa el costo sin ganancia de precisión.

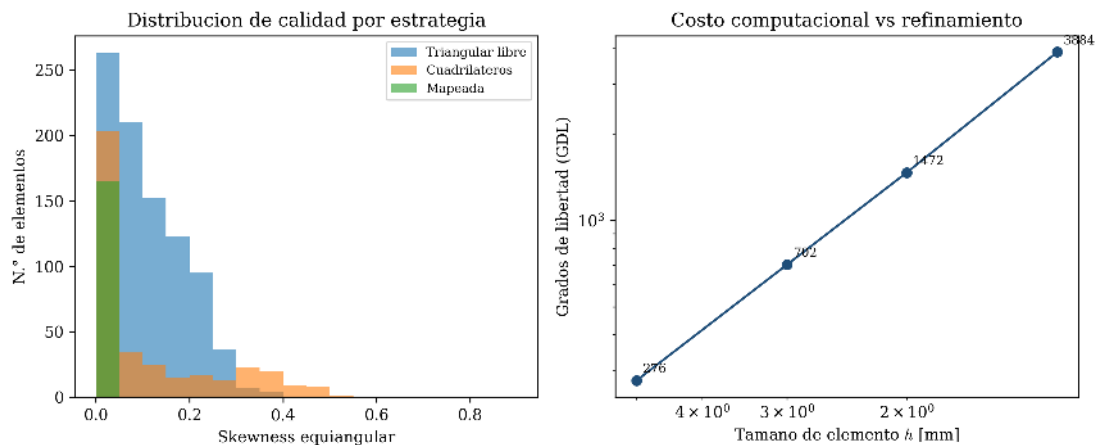


Figura 63: Izquierda: distribución de skewness por estrategia; la malla mapeada concentra todos sus elementos en calidad casi perfecta. Derecha: crecimiento de los GDL al refinar la malla (reducir h), en escala log–log.

8.7. Condiciones de frontera

Las condiciones de frontera (BC) cierran el problema (229) y lo vuelven no singular. En STATIC STRUCTURAL se distinguen dos clases:

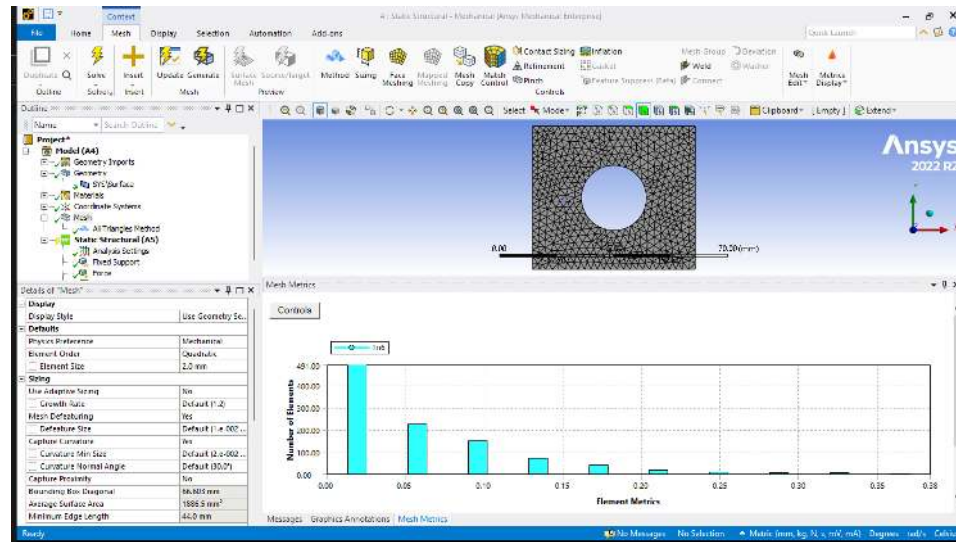


Figura 64: Selección del método de mallado en ANSYS (Mesh → Method). Las opciones corresponden a las estrategias de la Ec. 220 ilustradas en Python: libre (Tetrahedrons), mapeada (MultiZone) y barrida (Sweep).

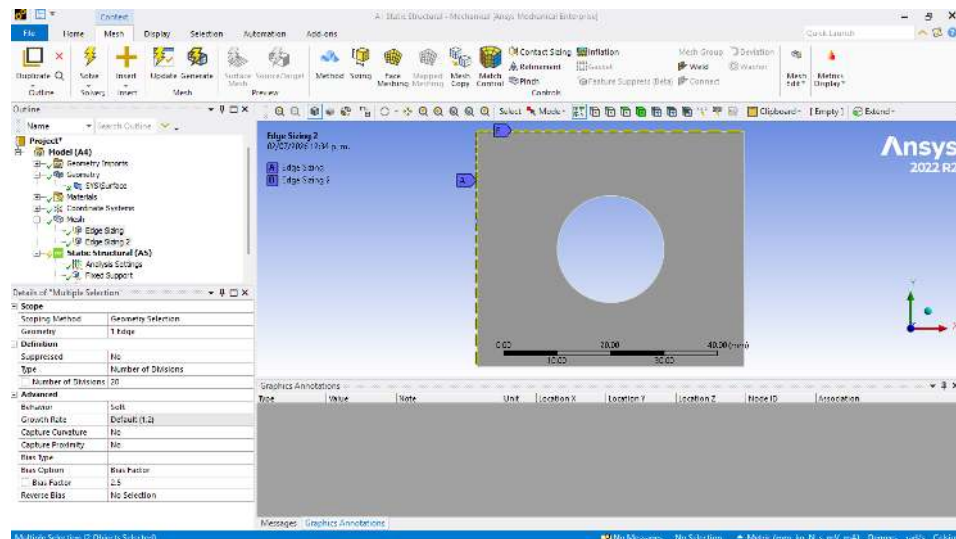


Figura 65: Control de sesgo en ANSYS (Edge Sizing → Bias Factor). El agrupamiento de nodos hacia el barreno reproduce la malla con sesgo de Figura 62.

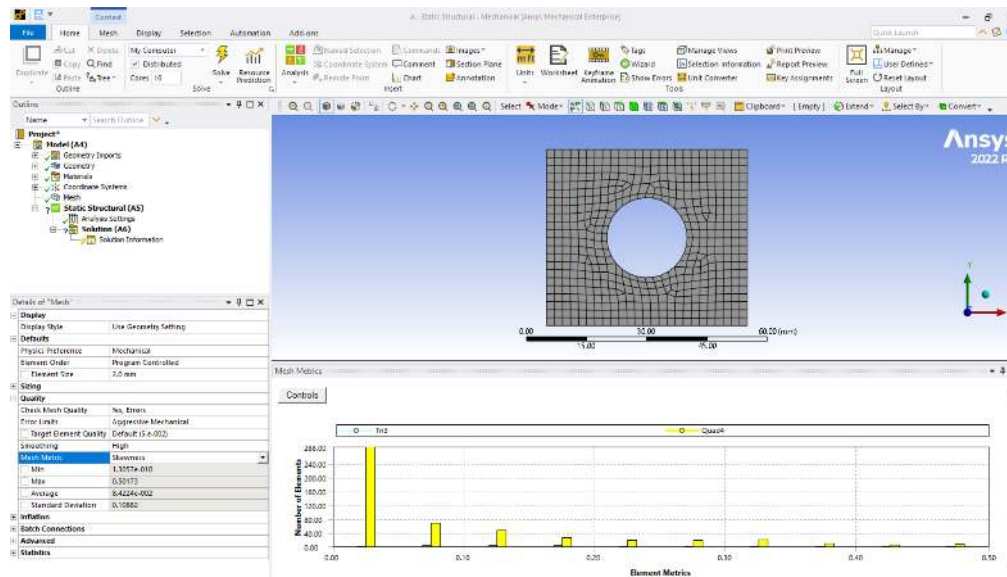


Figura 66: Evaluación de calidad en ANSYS (Mesh Metric). El gráfico de barras y los estadísticos son el análogo del histograma de skewness de Figura 63 y de Tabla 45.

- **Dirichlet (esenciales):** prescriben desplazamientos. En ANSYS: *Fixed Support* ($\mathbf{u} = \mathbf{0}$), *Displacement* (componentes prescritas), *Frictionless Support* (desplazamiento normal nulo, usado para simetría).
- **Neumann (naturales):** prescriben tracciones o fuerzas. En ANSYS: *Force*, *Pressure*, *Moment*. Se incorporan directamente en el vector $\mathbf{F}$ de (228) y no reducen GDL.

Una condición imprescindible es **eliminar los modos de cuerpo rígido**: en 2D hay tres (dos traslaciones y una rotación); en 3D, seis. Si no se restringen, $\mathbf{K}$ es singular y el solver falla. Las BC Dirichlet deben restringir al menos esos modos.

Advertencia: idealización de las BC. Las condiciones de frontera en FEA son siempre una idealización. En la realidad: (i) los apoyos no son puntos sino superficies de contacto con fricción; (ii) las cargas no son puntuales sino distribuidas sobre un área finita; (iii) puede haber holgura o desalineamiento. Estas diferencias son una fuente legítima de discrepancia experimento–modelo y deben documentarse. En el ensayo de flexión a tres puntos del curso, esto se traduce en modelar los apoyos como *pin* (restricción vertical y lateral) y *roller* (restricción vertical, libre axial), con la carga central como fuerza aplicada.

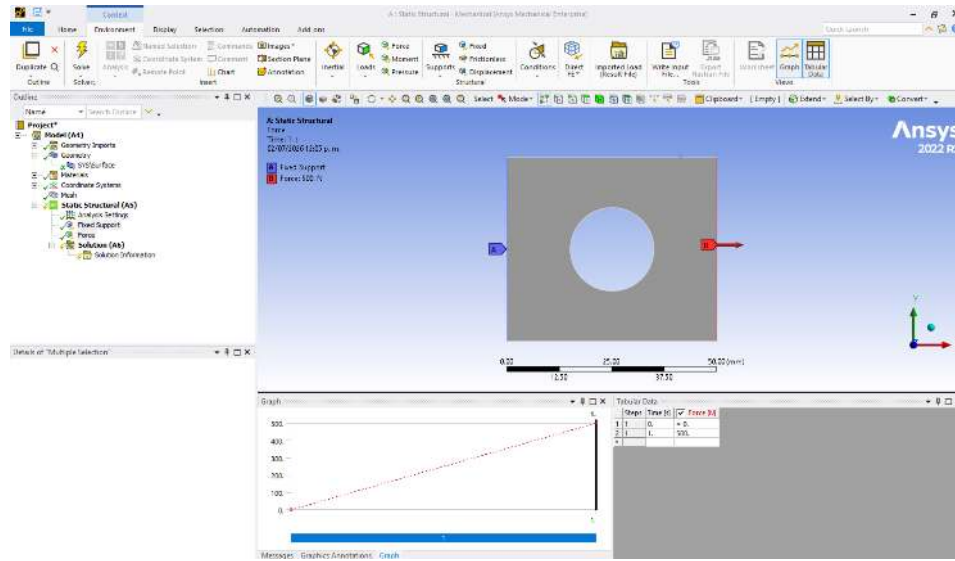


Figura 67: Imposición de condiciones de frontera en ANSYS. El *Fixed Support* restringe los modos de cuerpo rígido y la *Force* alimenta el vector $\mathbf{F}$ de la Ec. 229.

8.8. Flujo de trabajo en ANSYS Workbench – Static Structural

El módulo STATIC STRUCTURAL organiza el análisis en una secuencia de celdas (*Project Schematic*) que reproduce, en interfaz gráfica, los pasos formales de las secciones anteriores:

1. **Engineering Data:** definir el material (aquí Al 6061-T6: $E = 68.9 \text{ GPa}$, $\nu = 0.33$) $\Rightarrow$ tensor $\mathbf{D}$ de (217).
2. **Geometry:** construir/importar la geometría (Figura 55) $\Rightarrow$ dominio Ω .
3. **Model (Mesh):** generar la malla y controlar método, sizing, bias y orden de elemento $\Rightarrow$ discretización Ω_h .
4. **Setup (Static Structural):** imponer BC (Figura 67) $\Rightarrow$ cerrar (229).
5. **Solution:** resolver $\mathbf{KU} = \mathbf{F} \Rightarrow$ desplazamientos.
6. **Results:** recuperar y visualizar ε , σ , deformación total.

8.9. Aplicación a los proyectos doctorales

Las decisiones de mallado y de tipo de elemento dependen de la geometría y la física de cada problema. La Tabla 46 vincula las familias de geometría presentes en los proyectos del grupo con la estrategia de mallado y las condiciones de frontera recomendadas en STATIC STRUCTURAL.

Tres criterios transversales:

- **Explotar simetría.** Losas y secciones periódicas admiten modelar solo una celda o cuadrante con *Frictionless Support* en los planos de simetría, reduciendo GDL

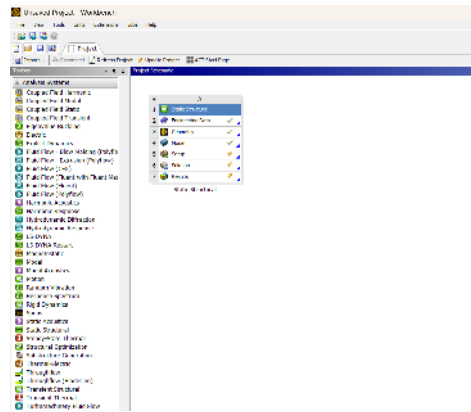


Figura 68: Esquema de proyecto de STATIC STRUCTURAL. Cada celda corresponde a un paso formal del FEM: material, dominio, malla, BC, solución y posproceso.

Tabla 46: Guía de mallado por familia de geometría/proyecto.

Familia de proyecto	Geometría típica	Malla recomendada	BC / carga
Soportes y fijaciones de sensores	Placa/brida con barreno	Mapeada + sesgo al barreno (o Quad)	Fixed Support + Force
Bancos de bombas centrífugas	Carcasa/voluta, brida de barrenos	Sweep en secciones de revolución	Presión interna + Fixed
Cables OPGW (sección transversal)	Múltiples hilos circulares (arcos)	Tetraédrica libre + refinamiento local	Tensión axial + térmica
Estructuras UAV	Placas con aligeramientos	Cáscara (SHELL181) + Quad	Empotramiento + carga distribuida
Losas reticulares (waffle)	Celda unitaria con casetones	Mapeada de celda + simetría	Frictionless (simetría) + presión

sin pérdida de exactitud.

- **Refinar donde hay gradiente.** Barrenos, filetes y cambios de sección concentran esfuerzos; ahí el sesgo o el *sizing* local (como en [Figura 62](#)) es obligatorio.
- **Elegir el orden del elemento.** Para fronteras curvas (secciones OPGW, voluta de bomba) los elementos cuadráticos capturan la geometría con menos elementos; para análisis no lineales, los lineales refinados suelen ser más económicos.

8.10. Anexo A: Código Python (Google Colab)

El script `Semana08_mallado_2D.py` genera la geometría, las cuatro mallas, las funciones de forma y las métricas de calidad reportadas en [Tabla 45](#). Requiere `gmsh`, `numpy` y `matplotlib`. En Google Colab, ejecutar primero:

```
!apt-get -qq install -y libglu1-mesa
!pip -q install gmsh
```

```
1 # -*- coding: utf-8 -*-
2 """
3 =====
4 Semana 8 - Metodo de Elementos Finitos I
5 Demostración de generación de mallas 2D con contorno curvo.
6 Curso: Prototipado y Simulación (DITA) - Universidad La Salle Bajío
7
8 Objetivo del script:
9     A partir de UNA geometría 2D con curvas (placa cuadrada con barreno
10     circular central, benchmark de concentración de esfuerzos), generar y
11     comparar las estrategias de mallado que ofrece un preprocesador tipo
12     ANSYS Workbench (Static Structural):
13         - Malla libre triangular (Frontal-Delaunay) -> Patch Conforming
14         - Malla de cuadriláteros (recombinación) -> Quad Dominant
15         - Malla mapeada estructurada (transfinita) -> Mapped / Sweep
16         - Malla mapeada con sesgo radial (bias) -> Bias Factor
17         - Elementos lineales vs cuadráticos (nodos/GDL)
18         - Funciones de forma  $N_i$ 
19         - Métricas de calidad (skewness, aspect ratio) y conteo de GDL
20
21 Ejecución en Google Colab:
22     !apt-get -qq install -y libglu1-mesa
23     !pip -q install gmsh
24     Luego ejecutar este archivo por celdas (marcadas con # %%).
25
26 Convenciones de reproducibilidad del curso:
27     matplotlib 'Agg', figuras 300 dpi en ./Figuras, semilla fija,
28     valores citados en el texto provienen de la corrida real.
29 =====
30 """
31
32 # %% Celda 0 - Configuración global -----
33 import os
34 import numpy as np
35 import matplotlib
36 matplotlib.use("Agg") # render sin pantalla (headless)
37 import matplotlib.pyplot as plt
```

```

38 from matplotlib.collections import PolyCollection, LineCollection
39 import gmsh
40
41 SEED = 8
42 rng = np.random.default_rng(SEED)          # reproducibilidad
43 os.makedirs("Figuras", exist_ok=True)
44
45 plt.rcParams.update({
46     "figure.dpi": 120, "savefig.dpi": 300,
47     "font.family": "serif", "font.size": 10,
48     "axes.grid": False, "mathtext.fontset": "cm",
49 })
50
51 # Parametros geometricos de la placa perforada [mm]
52 W = 50.0          # lado de la placa (dominio cuadrado)
53 A_HOLE = 10.0     # radio del barreno central
54 # Material de referencia (Al 6061-T6, usado en el hilo DIC-FEA del curso)
55 E_MAT, NU_MAT = 68.9e3, 0.33 # MPa, adimensional
56
57 RESULTS = {}      # acumulador de salidas numericas para el reporte final
58
59
60 # %% Celda 1 - Utilidades de extraccion y calidad de malla -----
61 def extract_mesh_2d():
62     """Devuelve coordenadas y conectividad (solo nodos de esquina) de la
63     malla 2D activa en gmsh, separando triangulos y cuadrilateros."""
64     ntags, ncoords, _ = gmsh.model.mesh.getNodes()
65     ncoords = np.array(ncoords).reshape(-1, 3)
66     tag2idx = {int(t): i for i, t in enumerate(ntags)}
67     xy = ncoords[:, :2]
68
69     tris, quads = [], []
70     etypes, _, enodes = gmsh.model.mesh.getElements(dim=2)
71     for et, nod in zip(etypes, enodes):
72         props = gmsh.model.mesh.getElementProperties(et)
73         nn = props[3]          # nodos por elemento
74         conn = np.array(nod).reshape(-1, nn)
75         corners = 3 if props[0].startswith("Triangle") else 4
76         for row in conn:
77             idx = [tag2idx[int(t)] for t in row[:corners]]
78             (tris if corners == 3 else quads).append(idx)
79     return xy, tris, quads
80
81
82 def polygon_angles(pts):
83     """Angulos internos (grados) de un poligono convexo ordenado."""
84     n = len(pts)
85     ang = []
86     for i in range(n):
87         a = pts[(i - 1) % n] - pts[i]
88         b = pts[(i + 1) % n] - pts[i]
89         c = np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b) + 1e-30)
90         ang.append(np.degrees(np.arccos(np.clip(c, -1, 1))))
91     return np.array(ang)
92
93
94 def equiangular_skewness(pts, ideal):
95     """Skewness equiangular (0 = ideal, 1 = degenerado). ideal=60 tri, 90 quad."""
96     ang = polygon_angles(pts)
97     tmax, tmin = ang.max(), ang.min()

```

```

98     return max((tmax - ideal) / (180 - ideal), (ideal - tmin) / ideal)
99
100
101 def aspect_ratio(pts):
102     """Relacion de aspecto = arista mayor / arista menor."""
103     n = len(pts)
104     L = [np.linalg.norm(pts[(i + 1) % n] - pts[i]) for i in range(n)]
105     return max(L) / (min(L) + 1e-30)
106
107
108 def mesh_quality(xy, tris, quads):
109     """Estadísticos de skewness y aspect ratio de la malla."""
110     sk, ar = [], []
111     for e in tris:
112         p = xy[e]; sk.append(equiangular_skewness(p, 60)); ar.append(aspect_ratio(p))
113     for e in quads:
114         p = xy[e]; sk.append(equiangular_skewness(p, 90)); ar.append(aspect_ratio(p))
115     sk, ar = np.array(sk), np.array(ar)
116     return dict(n_nodes=len(xy), n_elem=len(tris) + len(quads),
117               n_tri=len(tris), n_quad=len(quads),
118               sk_max=sk.max(), sk_mean=sk.mean(),
119               ar_max=ar.max(), ar_mean=ar.mean(),
120               skew=sk, ndof=2 * len(xy))
121
122
123 def plot_mesh(xy, tris, quads, title, fname, color_by_skew=False):
124     fig, ax = plt.subplots(figsize=(5.2, 5.0))
125     polys, vals = [], []
126     for e in tris + quads:
127         p = xy[e]; polys.append(p)
128         if color_by_skew:
129             ideal = 60 if len(e) == 3 else 90
130             vals.append(equiangular_skewness(p, ideal))
131     if color_by_skew:
132         pc = PolyCollection(polys, array=np.array(vals), cmap="RdYlGn_r",
133                           edgecolors="k", linewidths=0.25)
134         pc.set_clim(0, 0.9)
135         ax.add_collection(pc)
136         cb = fig.colorbar(pc, ax=ax, fraction=0.046, pad=0.04)
137         cb.set_label("Skewness equiangular")
138     else:
139         pc = PolyCollection(polys, facecolors="#cfe3f7",
140                           edgecolors="#1f4e79", linewidths=0.4)
141         ax.add_collection(pc)
142     ax.set_aspect("equal"); ax.autoscale_view()
143     ax.set_title(title, fontsize=10)
144     ax.set_xlabel("x [mm]"); ax.set_ylabel("y [mm]")
145     fig.tight_layout(); fig.savefig(f"Figuras/{fname}.png"); plt.close(fig)
146
147
148 # %% Celda 2 - Geometria base: placa con barreno (contorno curvo) -----
149 def build_plate(lc, order=1, quad=False, algo=6):
150     """Placa WxW con barreno circular central de radio A_HOLE (kernel OCC).
151     lc: tamaño de elemento; order: 1/2; quad: recombinar; algo: alg. 2D."""
152     gmsh.initialize()
153     gmsh.option.setNumber("General.Terminal", 0)
154     gmsh.model.add("placa")
155     occ = gmsh.model.occ
156     rect = occ.addRectangle(-W/2, -W/2, 0, W, W)
157     disk = occ.addDisk(0, 0, 0, A_HOLE, A_HOLE)

```

```

158     occ.cut([(2, rect)], [(2, disk)])
159     occ.synchronize()
160     gmsh.option.setNumber("Mesh.MeshSizeMin", lc)
161     gmsh.option.setNumber("Mesh.MeshSizeMax", lc)
162     gmsh.option.setNumber("Mesh.Algorithm", algo)
163     if quad:
164         gmsh.option.setNumber("Mesh.RecombineAll", 1)
165         gmsh.option.setNumber("Mesh.RecombinationAlgorithm", 1) # Blossom
166     gmsh.option.setNumber("Mesh.ElementOrder", order)
167     gmsh.model.mesh.generate(2)
168     xy, tris, quads = extract_mesh_2d()
169     q = mesh_quality(xy, tris, quads)
170     gmsh.finalize()
171     return xy, tris, quads, q
172
173
174 # Geometria (croquis) para figura de referencia
175 def plot_geometry():
176     th = np.linspace(0, 2*np.pi, 200)
177     fig, ax = plt.subplots(figsize=(5.2, 5.0))
178     ax.plot([-W/2, W/2, W/2, -W/2, -W/2],
179            [-W/2, -W/2, W/2, W/2, -W/2], "-", color="#1f4e79", lw=1.6)
180     ax.plot(A_HOLE*np.cos(th), A_HOLE*np.sin(th), "-", color="#c0392b", lw=1.6)
181     ax.annotate("", xy=(W/2, -W/2-4), xytext=(-W/2, -W/2-4),
182               arrowprops=dict(arrowstyle="<->", color="k"))
183     ax.text(0, -W/2-7, f"W = {W:.0f} mm", ha="center")
184     ax.annotate("", xy=(A_HOLE, 0), xytext=(0, 0),
185               arrowprops=dict(arrowstyle="->", color="#c0392b"))
186     ax.text(A_HOLE/2, 1.5, f"a = {A_HOLE:.0f} mm", color="#c0392b")
187     ax.text(0, W/2+3, "Contorno curvo (barreno)", ha="center",
188           color="#c0392b", fontsize=9)
189     ax.set_aspect("equal"); ax.set_xlim(-W/2-10, W/2+10); ax.set_ylim(-W/2-12, W/2+8)
190     ax.set_xlabel("x [mm]"); ax.set_ylabel("y [mm]")
191     ax.set_title("Geometria base: placa con barreno (benchmark $K_t$)", fontsize=10)
192     fig.tight_layout(); fig.savefig("Figuras/s8_geometria.png"); plt.close(fig)
193
194
195 plot_geometry()
196
197 # Malla libre triangular
198 xy, t, q, Q_TRI = build_plate(lc=2.5, order=1, quad=False, algo=6)
199 plot_mesh(xy, t, q, "Malla libre triangular (Frontal-Delaunay)", "s8_malla_libre_tri")
200 RESULTS["tri"] = Q_TRI
201
202 # Malla de cuadrilateros (recombinada)
203 xy, t, q, Q_QUAD = build_plate(lc=2.5, order=1, quad=True, algo=8)
204 plot_mesh(xy, t, q, "Malla de cuadrilateros (recombinacion Blossom)", "s8_malla_quad")
205 RESULTS["quad"] = Q_QUAD
206
207
208 # %% Celda 3 - Malla mapeada (transfinita) y sesgo (bias) -----
209 def bias_to_progression(bias_factor, n_nodes):
210     """Convierte el Bias Factor de ANSYS (razon arista mayor/menor total)
211     al coeficiente de Progression de gmsh (razon entre elementos
212     consecutivos). n_nodes = nodos sobre la arista -> (n_nodes-1) elementos.
213     ANSYS: BF = r^(nEl-1) => r = BF^(1/(nEl-1))."""
214     n_el = n_nodes - 1
215     if bias_factor <= 1.0 or n_el < 2:
216         return 1.0
217     return bias_factor ** (1.0 / (n_el - 1))

```

```

218
219
220 def build_annular_sector(nr, nt, order=1, bias=1.0, b=W/2):
221     """Sector anular de 90 grados entre r=A_HOLE y r=b, malla transfinita
222     (estructurada) de cuadrilateros. bias = Bias Factor estilo ANSYS
223     (razon arista mayor/menor). Se convierte a Progression de gmsh."""
224     r_prog = bias_to_progression(bias, nr)
225     gmsh.initialize()
226     gmsh.option.setNumber("General.Terminal", 0)
227     gmsh.model.add("anillo")
228     geo = gmsh.model.geo
229     a = A_HOLE
230     c = geo.addPoint(0, 0, 0)
231     p1 = geo.addPoint(a, 0, 0)      # inicio arco interno
232     p2 = geo.addPoint(0, a, 0)      # fin arco interno
233     p3 = geo.addPoint(0, b, 0)      # fin arco externo
234     p4 = geo.addPoint(b, 0, 0)      # inicio arco externo
235     l_rad1 = geo.addLine(p1, p4)     # radial inferior (a -> b)
236     a_ext = geo.addCircleArc(p4, c, p3) # arco externo
237     l_rad2 = geo.addLine(p3, p2)     # radial superior (b -> a)
238     a_int = geo.addCircleArc(p2, c, p1) # arco interno
239     cl = geo.addCurveLoop([l_rad1, a_ext, l_rad2, a_int])
240     s = geo.addPlaneSurface([cl])
241     geo.synchronize()
242     # Transfinito: nt nodos circunferenciales, nr radiales con progresion=bias
243     for arc in (a_ext, a_int):
244         geo.mesh.setTransfiniteCurve(arc, nt)
245     geo.mesh.setTransfiniteCurve(l_rad1, nr, "Progression", r_prog)
246     geo.mesh.setTransfiniteCurve(l_rad2, nr, "Progression", -r_prog) # sentido inverso
247     geo.mesh.setTransfiniteSurface(s)
248     geo.mesh.setRecombine(2, s)      # cuadrilateros
249     geo.synchronize()
250     gmsh.option.setNumber("Mesh.ElementOrder", order)
251     gmsh.model.mesh.generate(2)
252     xy, tris, quads = extract_mesh_2d()
253     Q = mesh_quality(xy, tris, quads)
254     gmsh.finalize()
255     return xy, tris, quads, Q
256
257
258 # Mapeada uniforme
259 xy, t, q, Q_MAP = build_annular_sector(nr=12, nt=16, bias=1.0)
260 plot_mesh(xy, t, q, "Malla mapeada estructurada (transfinita, bias=1.0)",
261           "s8_malla_mapeada")
262 RESULTS["map"] = Q_MAP
263
264 # Mapeada con sesgo radial -> comparacion 2 paneles
265 xyb, tb, qb, Q_BIAS = build_annular_sector(nr=12, nt=16, bias=6.0)
266 RESULTS["bias"] = Q_BIAS
267
268 fig, axs = plt.subplots(1, 2, figsize=(9.4, 4.6))
269 for ax, (X, TT, QQ, ttl) in zip(
270     axs, [(xy, t, q, "bias = 1.0 (uniforme)",
271            (xyb, tb, qb, "bias = 6.0 (denso hacia el barreno))]):
272     polys = [X[e] for e in TT + QQ]
273     ax.add_collection(PolyCollection(polys, facecolors="#cfe3f7",
274                                     edgecolors="#1f4e79", linewidths=0.4))
275     ax.set_aspect("equal"); ax.autoscale_view()
276     ax.set_title(ttl, fontsize=10); ax.set_xlabel("x [mm]"); ax.set_ylabel("y [mm]")
277 fig.suptitle("Efecto del sesgo (Bias Factor) sobre la distribucion radial de nodos",

```

```

278         fontsize=11)
279 fig.tight_layout(); fig.savefig("Figuras/s8_malla_bias.png"); plt.close(fig)
280
281
282 # %% Celda 4 - Elementos: nodos y grados de libertad (lineal vs cuadrático) --
283 def draw_element(ax, corners, mids, title):
284     poly = plt.Polygon(corners, closed=True, fill=True,
285                        facecolor="#eef3fb", edgecolor="#1f4e79", lw=1.5)
286     ax.add_patch(poly)
287     ax.scatter(*corners.T, s=70, color="#1f4e79", zorder=3, label="nodo esquina")
288     for i, p in enumerate(corners):
289         ax.annotate(f"{i+1}", p, textcoords="offset points",
290                    xytext=(6, 6), fontsize=9, color="#1f4e79")
291     if len(mids):
292         ax.scatter(*np.array(mids).T, s=55, marker="s", color="#c0392b",
293                    zorder=3, label="nodo intermedio")
294     ndof = 2 * (len(corners) + len(mids))
295     ax.set_title(f"{title}\n{len(corners)+len(mids)} nodos, {ndof} GDL", fontsize=9)
296     ax.set_aspect("equal"); ax.axis("off")
297
298
299 tri = np.array([[0, 0], [1, 0], [0.0, 1.0]])
300 quad = np.array([[0, 0], [1, 0], [1, 1], [0, 1]])
301 def edge_mids(c):
302     return [ (c[i] + c[(i+1) % len(c)]) / 2 for i in range(len(c)) ]
303
304 fig, axs = plt.subplots(1, 4, figsize=(11.5, 3.4), constrained_layout=True)
305 draw_element(axs[0], tri, [], "Tri3 (lineal)")
306 draw_element(axs[1], tri, edge_mids(tri), "Tri6 (cuadrático)")
307 draw_element(axs[2], quad, [], "Quad4 (lineal)")
308 draw_element(axs[3], quad, edge_mids(quad), "Quad8 (cuadrático)")
309 for ax in axs:
310     ax.margins(0.2)
311     axs[1].legend(loc="lower center", bbox_to_anchor=(1.05, -0.22),
312                  ncol=2, fontsize=9, frameon=False)
313 fig.suptitle("Nodos y grados de libertad por elemento "
314             "(estructural 2D: $u_x, u_y$ por nodo)", fontsize=10)
315 fig.savefig("Figuras/s8_elementos_dof.png"); plt.close(fig)
316
317
318 # %% Celda 5 - Funciones de forma $N_i$ -----
319 xi = np.linspace(-1, 1, 200)
320 # 1D lineal
321 N1L, N2L = (1 - xi) / 2, (1 + xi) / 2
322 # 1D cuadrático (Lagrange)
323 N1Q = xi * (xi - 1) / 2
324 N2Q = (1 - xi**2)
325 N3Q = xi * (xi + 1) / 2
326 # 2D bilineal $Q_4$: $N_1 = 1/4 (1-xi)(1-eta)$
327 g = np.linspace(-1, 1, 60); XX, YY = np.meshgrid(g, g)
328 N1_2D = 0.25 * (1 - XX) * (1 - YY)
329
330 fig = plt.figure(figsize=(11, 3.4))
331 ax1 = fig.add_subplot(1, 3, 1)
332 ax1.plot(xi, N1L, label="$N_1$"); ax1.plot(xi, N2L, label="$N_2$")
333 ax1.set_title("Lineal 1D"); ax1.set_xlabel(r"$xi$"); ax1.legend(fontsize=8)
334 ax1.axhline(0, color="k", lw=0.4); ax1.axhline(1, color="k", lw=0.3, ls=":")
335 ax2 = fig.add_subplot(1, 3, 2)
336 for N, lb in [(N1Q, "$N_1$"), (N2Q, "$N_2$"), (N3Q, "$N_3$")]:
337     ax2.plot(xi, N, label=lb)

```

```

338 ax2.set_title("Cuadratica 1D"); ax2.set_xlabel(r"$\xi$"); ax2.legend(fontsize=8)
339 ax2.axhline(0, color="k", lw=0.4)
340 ax3 = fig.add_subplot(1, 3, 3, projection="3d")
341 ax3.plot_surface(XX, YY, N1_2D, cmap="viridis", edgecolor="none")
342 ax3.set_title(r"Bilineal Q4: $N_1(\xi, \eta)$")
343 ax3.set_xlabel(r"$\xi$"); ax3.set_ylabel(r"$\eta$")
344 fig.suptitle("Funciones de forma: particion de la unidad $\sum_i N_i = 1$",
345             fontsize=10)
346 fig.tight_layout(); fig.savefig("Figuras/s8_funciones_forma.png"); plt.close(fig)
347
348 # Verificacion numerica: particion de la unidad
349 part_unity_1D = np.allclose(N1Q + N2Q + N3Q, 1.0)
350 part_unity_2D = np.isclose(N1_2D.sum() / # promedio ~0.25 en Q4 -> suma total nodos=1
351                             N1_2D.size, 0.25, atol=1e-9)
352 RESULTS["part_unity"] = bool(part_unity_1D)
353
354
355 # %% Celda 6 - Convergencia (conteo de GDL y calidad) -----
356 lcs = [5.0, 3.0, 2.0, 1.2]
357 conv = []
358 for lc in lcs:
359     _, _, _, Qc = build_plate(lc=lc, order=1, quad=False, algo=6)
360     conv.append((lc, Qc["n_nodes"], Qc["n_elem"], Qc["ndof"], Qc["sk_mean"]))
361 RESULTS["conv"] = conv
362
363 # Histograma de skewness por estrategia + curva GDL vs lc
364 fig, axs = plt.subplots(1, 2, figsize=(10, 4.2))
365 bins = np.linspace(0, 0.9, 19)
366 axs[0].hist(RESULTS["tri"]["skew"], bins, alpha=0.6, label="Triangular libre")
367 axs[0].hist(RESULTS["quad"]["skew"], bins, alpha=0.6, label="Cuadrilateros")
368 axs[0].hist(RESULTS["map"]["skew"], bins, alpha=0.6, label="Mapeada")
369 axs[0].set_xlabel("Skewness equiangular"); axs[0].set_ylabel("N.\u00b0 de elementos")
370 axs[0].set_title("Distribucion de calidad por estrategia"); axs[0].legend(fontsize=8)
371 lc_arr = [c[0] for c in conv]; dof_arr = [c[3] for c in conv]
372 axs[1].loglog(lc_arr, dof_arr, "o-", color="#1f4e79")
373 axs[1].invert_xaxis()
374 axs[1].set_xlabel("Tamano de elemento $h$ [mm]")
375 axs[1].set_ylabel("Grados de libertad (GDL)")
376 axs[1].set_title("Costo computacional vs refinamiento")
377 for lc, dof in zip(lc_arr, dof_arr):
378     axs[1].annotate(f"{dof}", (lc, dof), textcoords="offset points",
379                    xytext=(4, 5), fontsize=8)
380 fig.tight_layout(); fig.savefig("Figuras/s8_calidad.png"); plt.close(fig)
381
382
383 # %% Celda 7 - Reporte numerico (valores para el documento LaTeX) -----
384 def line(): print("-" * 70)
385 line(); print("SEMANA 8 - RESULTADOS NUMERICOS DE MALLADO"); line()
386 print(f"Geometria: placa {W:.0f}x{W:.0f} mm, barreno r={A_HOLE:.0f} mm")
387 print(f"Material de referencia: A1 6061-T6, E={E_MAT/1e3:.1f} GPa, nu={NU_MAT}")
388 line()
389 hdr = (f"{'Estrategia':<22}{ 'Nodos':>7}{ 'Elem':>7}{ 'GDL':>7}"
390        f"{'Sk_max':>9}{ 'Sk_med':>9}{ 'AR_max':>10}")
391 print(hdr)
392 for key, name in [("tri", "Triangular libre"), ("quad", "Cuadrilateros"),
393                  ("map", "Mapeada uniforme"), ("bias", "Mapeada + bias 6.0")]:
394     Q = RESULTS[key]
395     print(f"{'name':<22}{Q['n_nodes']:>7}{Q['n_elem']:>7}{Q['ndof']:>7}"
396           f"{'Q['sk_max']:>9.3f}{Q['sk_mean']:>9.3f}{Q['ar_max']:>10.2f}")
397 line()

```



```

398 print("Estudio de refinamiento (malla triangular libre):")
399 print(f"{'h [mm]':>8}{'Nodos':>8}{'Elem':>8}{'GDL':>8}{'Sk_med':>9}")
400 for lc, nn, ne, nd, skm in conv:
401     print(f"{'lc':>8.1f}{'nn':>8}{'ne':>8}{'nd':>8}{'skm':>9.3f}")
402 line()
403 print(f"Funciones de forma 1D cuadraticas, particion de la unidad: {part_unity_1D}")
404 print(f"Suma N_i(Q4) consistente (=1 por nodo): {part_unity_2D}")
405 line()
406 print("Figuras generadas en ./Figuras :")
407 for f in sorted(os.listdir("Figuras")):
408     print("    ", f)
409 line()
    
```

Listing 49: Generación y comparación de mallas 2D (Semana 8).

8.11. Anexo B: Referencias

1. Zienkiewicz, O. C., Taylor, R. L., Zhu, J. Z. (2013). *The Finite Element Method: Its Basis and Fundamentals*, 7.^a ed. Butterworth-Heinemann. DOI: 10.1016/C2009-0-24909-9.
2. Hughes, T. J. R. (2000). *The Finite Element Method: Linear Static and Dynamic Finite Element Analysis*. Dover.
3. Bathe, K. J. (2014). *Finite Element Procedures*, 2.^a ed. Prentice Hall.
4. Cook, R. D., Malkus, D. S., Plesha, M. E., Witt, R. J. (2001). *Concepts and Applications of Finite Element Analysis*, 4.^a ed. Wiley.
5. Geuzaine, C., Remacle, J.-F. (2009). Gmsh: A three-dimensional finite element mesh generator with built-in pre- and post-processing facilities. *International Journal for Numerical Methods in Engineering*, 79(11), 1309–1331. DOI: 10.1002/nme.2579.
6. ANSYS, Inc. (2025). *Meshing User's Guide y Mechanical APDL Modeling and Meshing Guide*, Release 2025 R2.

8.12. Ejercicios

1. **(Convergencia, 30 %)** Sobre la geometría de la placa con barreno, ejecute el script del Anexo A con al menos cinco tamaños de elemento h y reporte la evolución de los GDL y del skewness medio. Discuta a partir de qué h se satisface el criterio de la Ec. 222 suponiendo que la cantidad de interés converge como h^2 .
2. **(Estrategia de mallado, 30 %)** Elija la familia de geometría de su proyecto (Tabla 46) y justifique, con las métricas de Tabla 44, qué método de mallado y qué orden de elemento usaría. Genere en Python una malla representativa y reporte skewness máximo y razón de aspecto.
3. **(ANSYS, 40 %)** Reproduzca en STATIC STRUCTURAL el modelo de la placa con

barreno: material Al 6061-T6, malla con *Mapped Face Meshing* y *Bias Factor* hacia el barreno, *Fixed Support* en una arista y *Force* en la opuesta. Entregue las capturas correspondientes a Figuras 64 to 67 y compare el histograma de skewness de ANSYS con el de Figura 63.

Tabla 47: Rúbrica de los ejercicios de la Semana 8.

Criterio	Peso	Nivel esperado
Rigor numérico	30 %	Valores provenientes de corridas reales; unidades y convergencia documentadas
Justificación de mallado	30 %	Elección coherente con métricas de calidad y física del problema
Ejecución en ANSYS	30 %	Modelo correcto, BC no singulares, capturas completas
Comunicación técnica	10 %	Redacción precisa, figuras y tablas referenciadas

Rúbrica de evaluación.